

Edge Intelligence

Video Classification Task

Ashish Nikam

25MML0015

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input,
decode_predictions

# Load pretrained model
model = MobileNetV2(weights='imagenet')

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        raise ValueError("Cannot open video")

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    step = max(1, total_frames // max_frames) if total_frames > 0 else 1

    for i in range(total_frames):
        ret, frame = cap.read()
        if not ret:
            break

        if i % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)

    if len(frames) >= max_frames:
```

```

        break

    cap.release()
    return np.array(frames)

def classify_video(video_path):
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    frames = preprocess_input(frames)
    predictions = model.predict(frames, verbose=0)

    avg_pred = np.mean(predictions, axis=0)
    top_preds = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

    print("\nTop Predictions:")
    for label in top_preds[0]:
        print(f"{label[1]} : {label[2]*100:.2f}%")

    print("\nFinal Prediction:", top_preds[0][0][1])

video_path = "/content/drive/MyDrive/026c7465-309f6d33.mp4"
classify_video(video_path)

Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0
_224.h5
14536120/14536120 ————— 0s 0us/step
Downloading data from
https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_in
dex.json
35363/35363 ————— 0s 0us/step

Top Predictions:
cab : 17.92%
traffic_light : 6.14%
streetcar : 5.89%
jinrikisha : 3.63%
limousine : 3.56%

Final Prediction: cab

# =====
#  Mount Google Drive

```

```

# =====
from google.colab import drive
drive.mount('/content/drive')

# =====
# 2 Install & Import Libraries
# =====
!pip install -q tensorflow opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input,
decode_predictions

# =====
# 3 Load Pretrained Model
# =====
print("Loading MobileNetV2 model...")
model = MobileNetV2(weights='imagenet')
print("Model loaded successfully!\n")

# =====
# 4 Frame Extraction Function
# =====
def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    step = max(1, total_frames // max_frames)

    count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)

```

```

        count += 1
        if len(frames) >= max_frames:
            break

    cap.release()
    return np.array(frames)

video_path = "/content/drive/MyDrive/026c7465-309f6d33.mp4"

# =====
# 6 Extract Frames
# =====
extracted_frames = extract_frames(video_path)

if len(extracted_frames) == 0:
    raise ValueError("No frames extracted. Check video path.")

print("Extracted Frames Shape:", extracted_frames.shape)
print("Data Type:", extracted_frames.dtype)

# =====
# 7 Display First 5 Frames
# =====
plt.figure(figsize=(15,5))
for i in range(min(5, len(extracted_frames))):
    plt.subplot(1,5,i+1)
    plt.imshow(extracted_frames[i])
    plt.title(f"Frame {i+1}")
    plt.axis("off")
plt.show()

# =====
# 8 Show Pixel Values (5x5 Region)
# =====
print("\nTop-left 5x5 Pixel Region of First Frame:")
print(extracted_frames[0, 0:5, 0:5, :])

# =====
# 9 Normalize (0-255 → 0-1)
# =====
normalized_frames = extracted_frames / 255.0

print("\nNormalized Range:")
print("Min:", normalized_frames.min())
print("Max:", normalized_frames.max())

# =====
# 10 Model Preprocessing

```

```
# =====
processed_frames = preprocess_input(extracted_frames)

# =====
# 11 Predict on Frames
# =====
print("\nRunning predictions...")
predictions = model.predict(processed_frames, verbose=0)

# Average predictions across frames
avg_prediction = np.mean(predictions, axis=0)

# Decode top 5 predictions
top_predictions = decode_predictions(np.expand_dims(avg_prediction, axis=0),
top=5)

print("\nTop 5 Predictions:")
for pred in top_predictions[0]:
    print(f"{pred[1]} : {pred[2]*100:.2f}%")

print("\nFinal Prediction:", top_predictions[0][0][1])

Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
Loading MobileNetV2 model...
Model loaded successfully!
```

Extracted Frames Shape: (20, 224, 224, 3)
Data Type: uint8



Top-left 5x5 Pixel Region of First Frame:

```
[[[ 60  67  53]
   [ 57  63  50]
   [125 131 119]
   [103 108 101]
   [ 99 103  98]]]
```

```
[[[ 55  61  48]
   [ 60  66  53]
   [ 83  89  77]
   [ 88  93  87]]]
```

```
[121 126 120]]
```

```
[[ 48  54  41]  
 [ 62  67  55]  
 [ 68  74  63]  
 [ 80  85  79]  
 [103 108 102]]
```

```
[[ 45  51  38]  
 [ 50  56  43]  
 [ 60  66  54]  
 [ 81  86  80]  
 [ 76  81  75]]
```

```
[[ 43  49  36]  
 [ 44  47  35]  
 [ 50  52  41]  
 [102 107 101]  
 [ 74  78  73]]]
```

Normalized Range:

Min: 0.0

Max: 1.0

Running predictions...

Top 5 Predictions:

cab : 17.92%

traffic_light : 6.14%

streetcar : 5.89%

jinrikisha : 3.63%

limousine : 3.56%

Final Prediction: cab

Display all extracted frames in one long horizontal strip

```
num_frames = len(extracted_frames)
```

```
plt.figure(figsize=(40, 4)) # Wide figure
```

```
for i in range(num_frames):  
    plt.subplot(1, num_frames, i + 1)  
    plt.imshow(extracted_frames[i])  
    plt.title(f"Frame {i+1}", fontsize=8)  
    plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

